

---

EXERCISES

# Exercise sheet 2

---

AI505 — Optimization

Simon Holm  
February, 2026



DEPARTMENT OF MATHEMATICS  
AND COMPUTER SCIENCE

## Contents

|                          |    |
|--------------------------|----|
| Exercise 1*              | 3  |
| Solution                 | 3  |
| Exercise 2 <sup>+</sup>  | 3  |
| Solution                 | 3  |
| Exercise 3 <sup>+</sup>  | 3  |
| Solution                 | 3  |
| Exercise 4 <sup>+</sup>  | 4  |
| Solution                 | 4  |
| Exercise 5 <sup>+</sup>  | 4  |
| Solution                 | 4  |
| Exercise 6 <sup>+</sup>  | 5  |
| Solution                 | 5  |
| Exercise 7*              | 11 |
| Solution                 | 11 |
| Exercise 8*              | 11 |
| Solution                 | 11 |
| Exercise 9 <sup>+</sup>  | 12 |
| Solution                 | 12 |
| Exercise 10*             | 13 |
| Solution                 | 13 |
| Exercise 11 <sup>+</sup> | 13 |
| Exercise 12*             | 13 |
| Solution                 | 14 |

### Exercise 1\*

Suppose we have a unimodal function defined on the interval  $[1, 32]$ . After three function evaluations of our choice, will we be able to narrow the optimum to an interval of at most length 10? Why or why not? How much more can we reduce with one further evaluation?

#### Solution

Yes

for a unimodal function we can use Fibonacci, that is  $\frac{b-a}{F_{n+1}}$

That is

$$\frac{|b-a|}{3} = 3.36$$

For one more evaluation

$$\frac{|b-a|}{3} = 1.76$$

### Exercise 2<sup>+</sup>

Give an example of a problem when Fibonacci search can be applied while the bisection method not.

#### Solution

If the function is unimodal but not differentiable then we can use Fibonacci but not bisection, because bisection needs gradients

### Exercise 3<sup>+</sup>

Suppose we have  $f(x) = \frac{x^2}{2} - x$ . Apply the bisection method to find an interval containing the minimizer of  $f$  starting with the interval  $[0, 1000]$ . Execute three steps of the algorithm (you can do this by hand or in Python).

#### Solution

For this we need to check both  $\text{sign}(f'(0)) \neq \text{sign}(f'(\frac{b-a}{2}))$  and  $\text{sign}(f'(\frac{b-a}{2})) \neq \text{sign}(f'(1000))$

$$\text{sign}(f'(0)) \neq \text{sign}(f'(500)) \Rightarrow \text{sign}(-1) \neq \text{sign}(499) \quad \text{TRUE}$$

$$\text{sign}(f'(500)) \neq \text{sign}(f'(1000)) \Rightarrow \text{sign}(499) \neq \text{sign}(999) \quad \text{FALSE}$$

Then we know the minimum must lie within  $[0, 500]$

Now repeat

$$\text{sign}(f'(0)) \neq \text{sign}(f'(250)) \Rightarrow \text{sign}(-1) \neq \text{sign}(249) \quad \text{TRUE}$$

$$\text{sign}(f'(250)) \neq \text{sign}(f'(500)) \Rightarrow \text{sign}(249) \neq \text{sign}(499) \quad \text{FALSE}$$

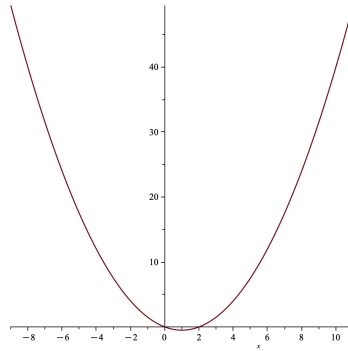
Now we know the minimum must lie within  $[0, 250]$

$$\text{sign}(f'(0)) \neq \text{sign}(f'(125)) \Rightarrow \text{sign}(-1) \neq \text{sign}(124) \quad \text{TRUE}$$

$$\text{sign}(f'(125)) \neq \text{sign}(f'(250)) \Rightarrow \text{sign}(124) \neq \text{sign}(249) \quad \text{FALSE}$$

After three steps the minimum must lie within  $[0, 125]$

This seems right considering the plot



### Exercise 4<sup>+</sup>

Suppose we have a function  $f(x) = (x + 2)^2$  on the interval  $[0, 1]$ . Is 2 a valid Lipschitz constant for  $f$  on that interval?

### Solution

For Lipschitz continuity on  $\ell = 2$

$$|f(x) - f(y)| \leq \ell(|x - y|), \forall x, y \in [a, b]$$

That would mean that

$$|f(0) - f(1)| \leq \ell(|0 - 1|) \quad |4 - 9| \leq 2 \quad \mathbf{FALSE}$$

So no

### Exercise 5<sup>+</sup>

The first Wolfe condition requires

$$f(x_k + \alpha d_k) \leq f(x_k) + \beta \alpha \nabla_{d_k} f(x_k)$$

What is the maximum step length  $\alpha$  that satisfies this condition, given that

$$f(x) = 5 + x_1^2 + x_2^2, x_k = [-1, -1], d_k = [1, 0] \text{ and } \beta = 10^{-4}$$

### Solution

We wish to find

$$\arg \max_{\alpha} f(x_k + \alpha d_k) \leq f(x_k) + \beta \alpha \nabla_{d_k} f(x_k).$$

That is the max  $\alpha$  where

$$f(x_k + \alpha d_k) = f(x_k) + \beta \alpha \nabla_{d_k} f(x_k).$$

That is

$$5 + (-1 + \alpha \cdot 1)^2 + (-1 + \alpha \cdot 0)^2 = 5 + (-1)^2 + (-1)^2 + 10^{-4} \cdot \alpha \nabla f(-1)^T d_k.$$

$$6 + 1 - 2\alpha + \alpha^2 = 5 + (-1)^2 + (-1)^2 + 10^{-4} \cdot \alpha \nabla f(-1)^T d_k.$$

$$7 - 2\alpha + \alpha^2 = 7 + 10^{-4} \cdot \alpha \cdot (-2).$$

We can solve this so  $\alpha = 0, 1.9998$ .

Then  $\alpha_{\max} = 1.9998$

## Exercise 6<sup>+</sup>

Consider the following function

$$f(x_1, x_2) = x_1^4 - 5x_1^2 + x_2^2$$

We are at  $x_0 = [0, 1]$  and we want to move in the descent direction  $d = [1, -1]$

- plot  $f(x_1, x_2)$
- Define the line search problem to solve
- Solve the line search problem using the Strong Bracketing Line Search Algorithm to find a value for the learning rate that satisfies the Strong Wolfe conditions.
- Plot the function of the line search and the iterates found in the process.

Note, you find the algorithm in Python in the slides. The focus of this exercise is on visualisation of the search process and on making sense of it.

### Solution

- Plotting  $f(x_1, x_2)$

```
import matplotlib.pyplot as plt
import numpy as np

def f(x1, x2):
    return x1**4 - 5*x1**2 + x2**2

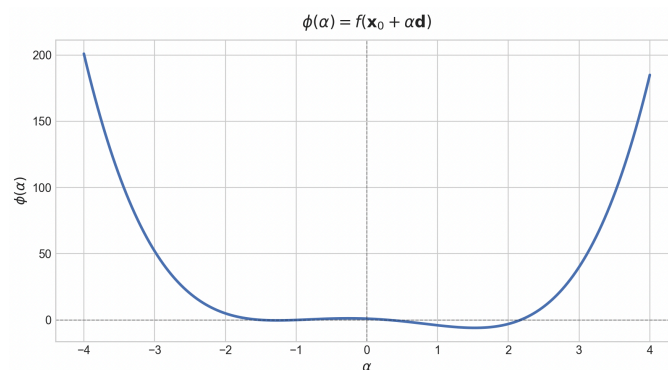
x0 = np.array([0.0, 1.0])
d = np.array([1.0, -1.0])

alpha = np.linspace(-4, 4, 400)
phi = f(x0[0] + alpha * d[0], x0[1] + alpha * d[1])

plt.style.use('seaborn-v0_8-whitegrid')

plt.plot(alpha, phi, color='#4C72B0', linewidth=2.5)
plt.axhline(0, color='black', linewidth=0.8, linestyle='--', alpha=0.4)
plt.axvline(0, color='black', linewidth=0.8, linestyle='--', alpha=0.4)

plt.set_title(r'$\phi(\alpha) = f(\mathbf{x}_0 + \alpha \mathbf{d})$', fontsize=15,
pad=12)
plt.set_xlabel(r'$\alpha$', fontsize=13)
plt.set_ylabel(r'$\phi(\alpha)$', fontsize=13)
plt.tick_params(labelsize=11)
plt.show()
```



- Defining the line search problem

Given a function  $\mathbb{R}^2 \rightarrow \mathbb{R}$ ,  
a current point  $x_k \in \mathbb{R}^2$ ,  
and a descent direction  $d_k \in \mathbb{R}^2$ .

Where

$$\phi(\alpha) = f(x_k + \alpha d_k).$$

Now the solve problem becomes

$$\alpha_k = \arg \min_{\alpha} \phi(\alpha) = \arg \min_{\alpha} f(x_k + \alpha d_k).$$

Since  $d_k$  is a descent direction, we only care about  $\alpha > 0$ .

Finally

$$\alpha_k = \arg \min_{\alpha > 0} f(x_k + \alpha d_k).$$

- Solving

We wish to solve the line search problem using the Strong Bracketing Line Search Algorithm to find a value for the learning rate that satisfies the Strong Wolfe conditions.

We can do this with the following code snippet, from lecture slides.

```
import autograd as ag
import autograd.numpy as np

def f(x):
    return x[0]**4 - 5*x[0]**2 + x[1]**2

nabla_f = ag.grad(f)

x0 = np.array([0.0, 1.0])
d = np.array([1.0, -1.0])

def strong_backtracking(f, nabla, x, d, alpha=1, beta=1e-4, sigma=0.1):
    y0, g0, y_prev, alpha_prev = f(x), nabla(x) @ d, None, 0
    alpha_lo, alpha_hi = None, None

    print("starting bracketing phase")

    # bracket phase
    while True:
        y = f(x + alpha*d)
        if y > y0 + beta*alpha*g0 or (y_prev is not None and y >= y_prev):
            alpha_lo, alpha_hi = alpha_prev, alpha
            print(f"low: {alpha_lo:.6f} high:{alpha_hi:.6f}")
            break
        dir_gradient = nabla(x + alpha*d) @ d
        if abs(dir_gradient) <= -sigma * g0:
            print(f"strong Wolfe in {alpha:.6f}")
            return alpha
        elif dir_gradient >= 0:
            alpha_lo, alpha_hi = alpha, alpha_prev
            print(f"bracket done: low={alpha_lo:.6f} high={alpha_hi:.6f}")
            break
```

```

        y_prev, alpha_prev, alpha = y, alpha, 2 * alpha

    print("starting zoom phase")

    # zoom phase
    ylo = f(x + alpha_lo*d)
    while abs(alpha_hi - alpha_lo) > 1e-10:
        alpha = (alpha_lo + alpha_hi)/2
        y = f(x + alpha*d)
        print(f"alpha={alpha:.6f}  f={y:.6f}  low={alpha_lo:.6f}
high={alpha_hi:.6f}")
        if y > y0 + beta *alpha*g0 or y >= ylo:
            alpha_hi = alpha
        else:
            g = nabla(x + alpha*d) @ d
            if abs(g) <= -sigma*g0:
                print(f"bracket done: lo={alpha_lo:.6f}  hi={alpha_hi:.6f}")
                print(f"Zoom done: lo={alpha_lo:.6f}  hi={alpha_hi:.6f}")
                return alpha
            elif g*(alpha_hi -alpha_lo) >= 0:
                alpha_hi = alpha_lo
            alpha_lo = alpha
    print(f"Zoom done: lo={alpha_lo:.6f}  hi={alpha_hi:.6f}")
    return alpha_lo # return the best found

alpha = strong_backtracking(f, nabla_f, x0, d)

print(alpha)

```

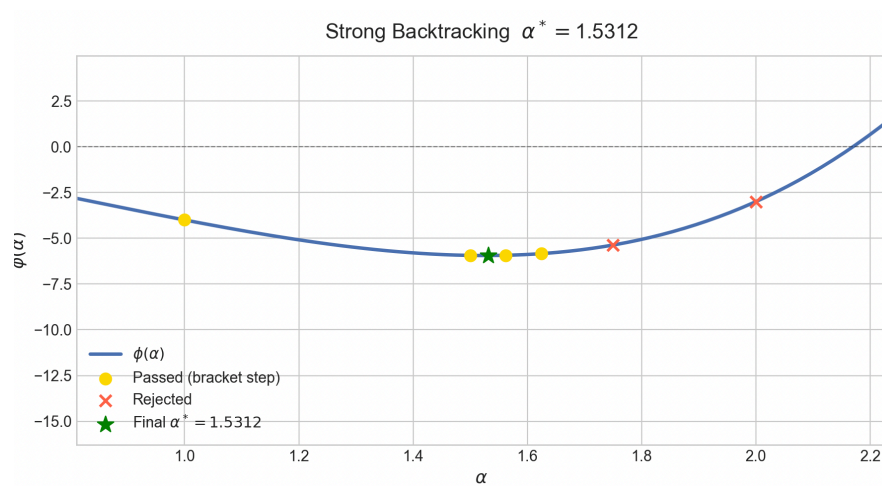
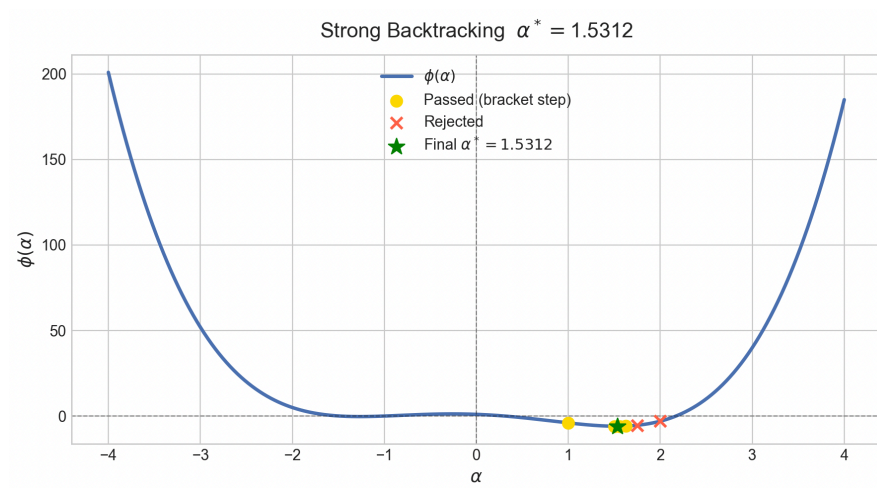
The output:

```

start
low: 1.000000  high:2.000000
alpha=1.500000  f=-5.937500  low=1.000000  high=2.000000
alpha=1.750000  f=-5.371094  low=1.500000  high=2.000000
alpha=1.625000  f=-5.839600  low=1.750000  high=1.500000
alpha=1.562500  f=-5.930161  low=1.625000  high=1.500000
alpha=1.531250  f=-5.943664  low=1.562500  high=1.500000
starting zoom phase
Zoom done: lo=1.562500  hi=1.500000
Accepted alpha: 1.53125

```

- Plotting the iterations of  $\alpha$



```
import autograd as ag
import autograd.numpy as anp
import matplotlib.pyplot as plt
import numpy as np

def f(x):
    return x[0]**4 - 5*x[0]**2 + x[1]**2

nabla_f = ag.grad(f)

x0 = anp.array([0.0, 1.0])
d = anp.array([1.0, -1.0])

def strong_backtracking(f, nabla, x, d, alpha=1, beta=1e-4, sigma=0.1):
    y0, g0, y_prev, alpha_prev = f(x), nabla(x) @ d, None, 0
    alpha_lo, alpha_hi = None, None
    rejected = [] # failed, shrink
    provisional = [] # passed, continue

    print("starting bracketing phase")

    # bracket phase
    while True:
```



```

y = f(x + alpha*d)
if y > y0 + beta*alpha*g0 or (y_prev is not None and y >= y_prev):
    rejected.append(alpha)
    alpha_lo, alpha_hi = alpha_prev, alpha
    print(f"low: {alpha_lo:.6f} high:{alpha_hi:.6f}")
    break
dir_gradient = nabla(x + alpha*d) @ d
if abs(dir_gradient) <= -sigma * g0:
    print("strong Wolfe in " r"$\alpha$" f"{alpha:.6f}")
    return alpha, rejected, provisional
elif dir_gradient >= 0:
    rejected.append(alpha)
    alpha_lo, alpha_hi = alpha, alpha_prev
    print(f"bracket done: low={alpha_lo:.6f} high={alpha_hi:.6f}")
    break
else:
    provisional.append(alpha) # sufficient decrease holds, keep expanding
    y_prev, alpha_prev, alpha = y, alpha, 2 * alpha

print("starting zoom phase")

# zoom phase
ylo = f(x + alpha_lo*d)
while abs(alpha_hi - alpha_lo) > 1e-10:
    alpha = (alpha_lo + alpha_hi)/2
    y = f(x + alpha*d)
    print(f"alpha={alpha:.6f} f={y:.6f} low={alpha_lo:.6f}
high={alpha_hi:.6f}")
    if y > y0 + beta*alpha*g0 or y >= ylo:
        rejected.append(alpha)
        alpha_hi = alpha
    else:
        g = nabla(x + alpha*d) @ d
        if abs(g) <= -sigma*g0:
            print(f"Zoom done: lo={alpha_lo:.6f} hi={alpha_hi:.6f}")
            return alpha, rejected, provisional
        elif g*(alpha_hi - alpha_lo) >= 0:
            rejected.append(alpha)
            alpha_hi = alpha_lo
        else:
            provisional.append(alpha) # updates alpha_lo, still zooming
            alpha_lo = alpha
    print(f"Zoom done: lo={alpha_lo:.6f} hi={alpha_hi:.6f}")
    return alpha_lo, rejected, provisional

accepted_alpha, rejected_alphas, provisional_alphas = strong_backtracking(f, nabla_f,
x0, d)

print(f"Accepted alpha: {accepted_alpha}")

# --- Plot ---
def phi_scalar(a):
    x = x0 + a * d
    return float(f(x))

alpha_set = np.linspace(-4, 4, 400)

```

```

phi_vals = np.array([phi_scalar(a) for a in alpha_set])

plt.style.use('seaborn-v0_8-whitegrid')
fig, ax = plt.subplots(figsize=(9, 5))

ax.plot(alpha_set, phi_vals, color='#4C72B0', linewidth=2.5, label=r'$\phi(\alpha)$')
ax.axhline(0, color='black', linewidth=0.8, linestyle='--', alpha=0.4)
ax.axvline(0, color='black', linewidth=0.8, linestyle='--', alpha=0.4)

# provisional: passed sufficient decrease, used to advance bracket
if provisional_alphas:
    prov = np.array(provisional_alphas)
    ax.scatter(prov, [phi_scalar(a) for a in prov],
               color='gold', zorder=5, s=70, label='Passed (bracket step)')

# rejected: failed sufficient decrease or curvature
if rejected_alphas:
    rej = np.array(rejected_alphas)
    ax.scatter(rej, [phi_scalar(a) for a in rej],
               color='tomato', zorder=5, s=70, marker='x', linewidths=2,
               label='Rejected')

# final accepted alpha
ax.scatter([accepted_alpha], [phi_scalar(accepted_alpha)],
           color='green', zorder=6, s=150, marker='*',
           label=f'Final $\alpha^* = \{accepted\_alpha:.4f\}$')

ax.set_title(r'Strong Backtracking $\alpha^* = $' + f'\{accepted\_alpha:.4f\}',
             fontsize=15, pad=12)
ax.set_xlabel(r'$\alpha$', fontsize=13)
ax.set_ylabel(r'$\phi(\alpha)$', fontsize=13)
ax.tick_params(labelsize=11)
ax.legend(fontsize=11)
plt.tight_layout()
plt.show()

```

## Exercise 7\*

The steepest descent algorithm is a Descent Direction Iteration method that moves along  $d_k = -\nabla f(x_k)$  at every step. Program steepest descent algorithms using the backtracking line search. Use them to minimize the Rosenbrock function. Set the initial step length  $\alpha_0 = 1$  and print the step length used by each method at each iteration. First, try the initial point  $x_0 = [1.2, 1.2]$  and then the more difficult starting point  $x_0 = [-1.2, 1]$ . Consider implementing and comparing also other ways for solving the line search problem and the conjugate gradient.

### Solution

- First with point  $x_0 = [1.2, 1.2]$

```
import autograd as auto
import autograd.numpy as np

def f(x):
    return (1-x[0])**2 + 100*(x[1]-x[0]**2)**2

nabla_f = auto.grad(f)

x0 = np.array([1.2, 1.2])
d = -(nabla_f(x0))

def backtracking_line_search(f, grad, x, d, alpha_0=1, p=0.5, beta=1e-4):
    y, g, alpha = f(x), grad(x), alpha_0

    while ( f(x + alpha * d) > y + beta * alpha * np.dot(g, d) ) :
        alpha *= p
    return alpha

def solve(max_iter=1000)

alpha = backtracking_line_search(f,nabla_f,x0,d)

print(alpha)

print("done!")
```

$$\alpha = 0.0009765625$$

starting from  $[-1.2, 1]$  gives the same value for  $\alpha$

## Exercise 8\*

Descent direction methods may use search directions other than the steepest descent mentioned in the previous exercise. In general, which descent direction guarantees to produce a decrease in  $f$ ?

### Solution

We want to show that

$$f(x_{k+1} + \alpha d_{k+1}) < f(x_k)$$

Then since

$$f(x + \alpha d_k) = f(x) + \alpha \nabla_{d_k} f(x_k)$$

For descent directions

$$\nabla f(x_k)^T d_k < 0$$

## Exercise 9<sup>+</sup>

Compute the gradient of  $x^T A x - b^T x$  when  $A$  is symmetric. You have the result in the slides of lectures, here you are asked to show that the result is correct.

### Solution

We are given the information that

$$f(x) = x^T A x - b^T x.$$

This is equivalent to

$$\sum_{i,j} x_i a_{ij} x_j - \sum_i b_i \cdot x_i.$$

Now we can differentiate to find the gradient  $\nabla f$

$$\frac{\partial}{\partial x} \sum_{i,j} x_i a_{ij} x_j - \frac{\partial}{\partial x} \sum_i b_i \cdot x_i = \sum_{i,j} \frac{\partial(x_i x_j)}{\partial x_k} - \frac{\partial}{\partial x_k} \sum_i b_i \cdot x_i.$$

We know that

- if  $i = k \neq j$  then  $\frac{\partial(x_i x_j)}{\partial x_k} = x_j$
- if  $i \neq k = j$  then  $\frac{\partial(x_i x_j)}{\partial x_k} = x_i$
- if  $i = k = j$  then  $\frac{\partial(x_i x_j)}{\partial x_k} = 2x_k$

therefore

$$\frac{\partial}{\partial x} (x^T A x - b^T x) = \sum_j x_j a_{kj} + \sum_i x_i a_{ik} - b_k.$$

We know that  $\sum a_{kj} x_j = (Ax)_k$  (dot product of the  $k$ 'th column)

and  $\sum a_{ik} x_i = (A^T x)_k$  (dot product of the  $k$ 'th row)

$$\sum_j x_j a_{kj} + \sum_i x_i a_{ik} - b_k \Rightarrow Ax + A^T x - b$$

Where  $A + A^T = 2A$  since  $A$  is symmetric

$$\frac{\partial}{\partial x} (x^T x - b^T x) = (A + A^T)x - b = 2Ax - b$$

or

$$2Ax = b$$

## Exercise 10\*

Apply one step of gradient descent to  $f(x) = x^4$  from  $x_0 = 1$  with both a unit step factor and with exact line search.

### Solution

Since  $f(x) = x^4$ ,  $x_0$  then  $f'(x) = 4x^3$

Gradient Descent step:

$$x_1 = x_0 - 1(4) = -3$$

Exact line search

$$\arg \min_{\alpha \geq 0} f(x - \alpha \nabla_{d_k} f(x_k)) = \arg \min_{\alpha \geq 0} (1 + \alpha \cdot (-4))$$

We can solve  $\alpha^* = \frac{1}{4}$

Then

$$x_1 = x_0 + \frac{1}{4} \cdot (-4) = 0$$

## Exercise 11<sup>+</sup>

Yet to do

## Exercise 12\*

Consider applying gradient descent to the one-dimensional quadratic function

$$f(x) = \frac{1}{2}x^2$$

with the update rule:

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

where  $\nabla f(x) = x$

Tasks

- Derive the update formula for  $x_k$
- Show that the error  $e_k = |x_k|$  follows an exponential decay when  $0 < \alpha < 2$ .
- Compute  $\frac{e_{k+1}}{e_k}$  and determine the rate of convergence for different values of  $\alpha$
- Set up a Python experiment where gradient descent is applied with different step sizes ( $\alpha$ ) and verify the theoretical convergence rate numerically. *Hint: Try  $\alpha = 0.1, 0.5, 1, 1.5$  and observe how quickly the errors decrease.*

## Solution

We know that

$$x_k = x_{k-1} + \alpha \cdot (-x_{k-1}) = (1 - \alpha)x_{k-1}$$

and since for each  $x$ ,  $|x_k| = e_k$  then

$$e_k = |x_k| = |(1 - \alpha)x_{k-1}| = |(1 - \alpha)| \cdot |e_{k-1}|$$

we can then say that

$$|e_k| = |(1 - \alpha)| \cdot |e_{k-1}| \Rightarrow \frac{|e_k|}{|e_{k-1}|} = |(1 - \alpha)|$$

The fact that the error  $e_{k-1} \rightarrow e_k$  is bounded by  $|(1 - \alpha)|$

Then we can say that

$$e_k = |(1 - \alpha)|^k \cdot e_0$$

Since we want the error to go towards 0, that is  $\lim_{k \rightarrow \infty} e_k = 0$

Then we want

$$\lim_{k \rightarrow \infty} |(1 - \alpha)|^k \rightarrow 0$$

For this

$$\alpha \in (0, 2)$$

This could be yet another constraint, ensuring that the step length converges to 0 error.

Let's compute  $\frac{e_{k+1}}{e_k}$  by  $\alpha$ -values  $-3, 0.5, 1, 1.5$  and  $3$ .

So we wish to compute

$$\lim_{k \rightarrow \infty} |1 - (-3)|^k \cdot e_0$$

We can test different  $\alpha$ 's using gradient descent

```
import autograd as auto
import autograd.numpy as np

def f(x):
    return 1/2 * x**2

nabla_f = auto.grad(f)

x0 = 1.0 # changed from 0 to avoid trivial starting point

def gradient_descent(f, grad, x0, alpha=0.1, tol=1e-6, max_iter=1000):
    x = x0
    for i in range(max_iter):
        g = grad(x)
        x_new = x - alpha * g
        if abs(x_new - x) < tol:
            print(f"Converged at iteration {i+1}")
            break
        x = x_new
    return alpha, x

alpha1, x_opt = gradient_descent(f, nabla_f, x0, 0.1)
print(f"With alpha: {alpha1}, optimal x: {x_opt:.6f}, f(x): {f(x_opt):.6f}")

alpha2, x_opt = gradient_descent(f, nabla_f, x0, 0.5)
print(f"With alpha: {alpha2}, optimal x: {x_opt:.6f}, f(x): {f(x_opt):.6f}")

alpha3, x_opt = gradient_descent(f, nabla_f, x0, 1)
print(f"With alpha: {alpha3}, optimal x: {x_opt:.6f}, f(x): {f(x_opt):.6f}")

alpha4, x_opt = gradient_descent(f, nabla_f, x0, 1.5)
print(f"With alpha: {alpha4}, optimal x: {x_opt:.6f}, f(x): {f(x_opt):.6f}")

alpha5, x_opt = gradient_descent(f, nabla_f, x0, 1.9)
print(f"With alpha: {alpha5}, optimal x: {x_opt:.6f}, f(x): {f(x_opt):.6f}")
```

Output:

```
Converged at iteration 111
With 0.1, optimal x: 0.000009, f(x): 0.000000
Converged at iteration 20
With 0.5, optimal x: 0.000002, f(x): 0.000000
Converged at iteration 2
With 1, optimal x: 0.000000, f(x): 0.000000
Converged at iteration 22
With 1.5, optimal x: -0.000000, f(x): 0.000000
Converged at iteration 139
With 1.9, optimal x: 0.000000, f(x): 0.000000
```

This fits well with the plot of

$$f(x) = \frac{1}{2}x^2$$

